

Compléments d'Algorithmique

Chapitre 6 : Interfaces graphiques

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 6. Interfaces graphiques

- 1 Petit exemple introductif
- 2 Positionner des widgets
- 3 Principaux widgets
- 4 Menus
- 5 Canvas
- 6 Events
- 7 Un exemple complet

Interfaces graphiques

Les **interfaces graphiques** (*Graphical User Interfaces, GUI*) permettent de créer des applications avec des fenêtres, des boutons, des menus, etc. et de gérer l'interaction avec l'utilisateur (via la souris, le clavier, etc.).

Il existe quantité de bibliothèques pour en créer, et les principes sont souvent similaires.

Nous allons nous concentrer sur la bibliothèque standard de Python, `tkinter`, et en montrer l'essentiel. Pour plus de détails, voir la documentation officielle.

<https://docs.python.org/3/library/tkinter.html>

Petit exemple introductif

Petit exemple introductif

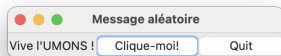
```
from tkinter import *
from tkinter import ttk
import random

messages = ["Vive l'UMONS", "Tkinter, c'est facile", "Clique encore"]
def changer_message(lbl):
    lbl.config(text=random.choice(messages))

root = Tk()
root.title("Message aléatoire")
mainframe = ttk.Frame(root, padding=10)
mainframe.grid()
label = ttk.Label(mainframe, text="Bienvenue !")
label.grid(column=0, row=0)
btn1 = ttk.Button(mainframe, text="Clique-moi!", \
    command=lambda: changer_message(label))
btn1.grid(column=1, row=0)
ttk.Button(mainframe, text="Quit", \
    command=root.destroy).grid(column=2, row=0)

root.mainloop()
```

Petit exemple introductif



Quelques brèves explications (nous reviendrons en détail sur ces concepts)

- `from tkinter import *` : importe tout depuis la librairie `tkinter` **standard** ;
- `from tkinter import ttk` : importe le module `ttk` de `tkinter` (*themed tk*) qui introduit (ou remplace par) des éléments plus modernes et qui s'adaptent au visuel des systèmes d'exploitation ;
- de cette manière, nous avons le choix : utiliser les éléments standards (comme `Label`, sans préfixe) ou leurs équivalents `ttk` (via `ttk.Label`) ;

Petit exemple introductif

- `root = Tk()` : crée une fenêtre principale ;
- `mainframe = ttk.Frame(root, ...)` : crée un cadre dans la fenêtre principale ;
- `label = ttk.Label(mainframe, ...)` : crée un label (e.g., du texte ou une image) dans le cadre ;
- `btn1 = ttk.Button(mainframe, ...)` : crée un bouton dans le cadre ;
- `element.grid(...)` : place l'élément dans une grille qui permet d'organiser la position des éléments ;
- `root.mainloop()` : lance la boucle principale de l'interface graphique, pour gérer les événements (e.g., clics de souris, touches du clavier). Ceci se place à la fin, une fois la GUI construite. C'est ce qui lui permet de "prendre vie".

Petit exemple introductif

Les éléments comme les cadres, les labels, les boutons, etc., sont des **widgets**.

Les **widgets** :

- ont des **attributs** (e.g., `text`, `command`) qui permettent de les configurer ;
- ont des **méthodes** (e.g., `config`, `grid`) qui permettent de les manipuler ;
- peuvent réagir à des **événements** (e.g., clic de souris) via des **callbacks** (e.g., `command`).

Positionner des widgets

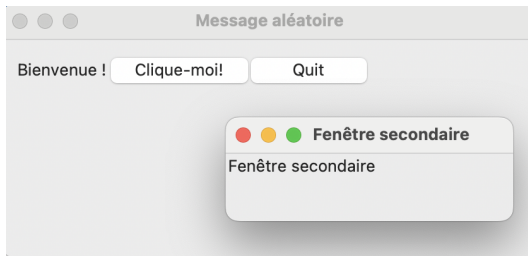
Fenêtres et cadres

Une fenêtre est créée via `root = Tk()` (`root` est ici un nom choisi pour signaler que c'est la fenêtre principale).

On peut aussi créer plusieurs fenêtres via `TopLevel()` (mais une seule est active à la fois).

```
(...)  
secondWindow = Toplevel()  
secondWindow.title("Fenêtre secondaire")  
  
label2 = ttk.Label(secondWindow, text="Fenêtre secondaire")  
label2.grid(column=0, row=0)  
(...)
```

Fenêtres et cadres



Fenêtres et cadres

On peut ajouter des cadres (**frames**) dans une fenêtre pour regrouper et organiser des widgets via `ttk.Frame(...)`.

Les cadres sont des éléments invisibles qui permettent de regrouper des widgets et de les organiser. Ils peuvent être imbriqués.

Pour **positionner des éléments** (widgets ou sous-cadres) dans un cadre, on utilise une des trois méthodes suivantes, que nous allons explorer : `grid`, `pack` ou `place`.

On peut rendre un cadre visible en lui ajoutant un contour (via l'option `borderwidth`) et/ou un relief (via l'option `relief`).

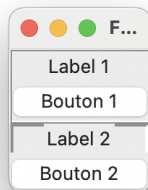
Fenêtres et cadres

```
frame1 = ttk.Frame(root, borderwidth=2, relief="ridge")
frame1.pack()
label1 = ttk.Label(frame1, text="Label 1")
label1.pack()
button1 = ttk.Button(frame1, text="Bouton 1")
button1.pack()

frame2 = ttk.Frame(root, relief="sunken")
frame2.pack()
label2 = ttk.Label(frame2, text="Label 2")
label2.pack()
button2 = ttk.Button(frame2, text="Bouton 2")
button2.pack()
```

La méthode `pack` place les éléments les uns en dessous des autres, dans la fenêtre ou le cadre qui les contient (parent).

Fenêtres et cadres



Pour le moment, le rendu n'est pas très beau car les cadres sont collés les uns aux autres. Nous pouvons améliorer cela en ajoutant des marges ([padding](#)).

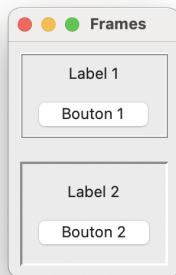
Options de pack

```
frame1 = ttk.Frame(root, borderwidth=2, relief="ridge")
frame1.pack(pady=10, padx=10, fill="both", expand=True)
label1 = ttk.Label(frame1, text="Label 1")
label1.pack(pady=5, padx=5)
button1 = ttk.Button(frame1, text="Bouton 1")
button1.pack(pady=5, padx=5)

frame2 = ttk.Frame(root, padding=10, relief="sunken")
frame2.pack(pady=10, padx=10, fill="both", expand=True)
label2 = ttk.Label(frame2, text="Label 2")
label2.pack(pady=5, padx=5)
button2 = ttk.Button(frame2, text="Bouton 2")
button2.pack(pady=5, padx=5)
```

- **pady et padx ajoutent des marges verticales et horizontales (en pixels) ;**

Options de pack



- `fill="both"` : remplit l'espace disponible en largeur et en hauteur ;
- `fill="x"` : remplit l'espace disponible en largeur ;
- `fill="y"` : remplit l'espace disponible en hauteur ;
- `expand=True` : permet d'agrandir le cadre si la fenêtre est redimensionnée.

Options de pack

D'autres options de pack peuvent être utiles.

Option	Effet
<code>side=TOP/LEFT/RIGHT/BOTTOM</code>	Positionne le widget dans le conteneur selon le côté indiqué.
<code>ipadx, ipady</code>	Ajoute un padding interne (agrandit l'espace à l'intérieur du widget).
<code>anchor="n/s/e/w/center"</code>	Positionne le widget dans l'espace disponible (ex : nord, sud, est, ouest, centre).

Voir exemple présenté en séance.

LabelFrame

Un `ttk.LabelFrame` est un cadre qui possède un titre (via l'option `text`).

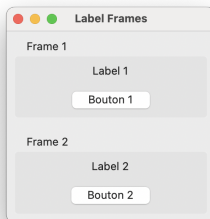
```
frame1 = ttk.LabelFrame(root, text="Frame 1")
frame1.pack(pady=10, padx=10, fill="both", expand=True)

label1 = ttk.Label(frame1, text="Label 1")
label1.pack(pady=5, padx=5)
button1 = ttk.Button(frame1, text="Bouton 1")
button1.pack(pady=5, padx=5)

frame2 = ttk.LabelFrame(root, text="Frame 2")
frame2.pack(pady=10, padx=10, fill="both", expand=True)

label2 = ttk.Label(frame2, text="Label 2")
label2.pack(pady=5, padx=5)
button2 = ttk.Button(frame2, text="Bouton 2")
button2.pack(pady=5, padx=5)
```

LabelFrame



Grid

La méthode `grid` permet de positionner les éléments dans une grille.

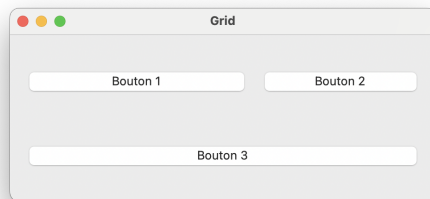
```
frame.columnconfigure(0, weight=2)
frame.columnconfigure(1, weight=1)
frame.rowconfigure(0, weight=1)
frame.rowconfigure(1, weight=1)

button1 = ttk.Button(frame, text="Bouton 1")
button2 = ttk.Button(frame, text="Bouton 2")
button3 = ttk.Button(frame, text="Bouton 3")

button1.grid(row=0, column=0, sticky="nsew")
button2.grid(row=0, column=1, sticky="nsew")
button3.grid(row=1, column=0, columnspan=2, sticky="nsew")
```

- `columnconfigure` et `rowconfigure` permettent à la colonne ou la ligne de s'étirer si la fenêtre grandit (ici la colonne 0 a un poids qui est le double de la colonne 1, elle s'étendra donc 2 fois plus) ;

Grid



- `sticky` permet de coller le widget aux bords de la cellule de la grille (ici, les boutons sont collés au nord, sud, est et ouest et prendront donc toute la place disponible) ;
- `columnspan` permet de fusionner plusieurs colonnes (ici, le bouton 3 s'étend sur les colonnes 0 et 1).

Place

La méthode `place` permet de positionner les éléments via des coordonnées et permet donc un contrôle précis du placement des widgets.

Elle utilise

- des coordonnées absolues (`x`, `y`) pour positionner le coin supérieur gauche du widget ;
- des options `relx`, `rely` pour positionner le widget de manière relative par rapport à la taille de la fenêtre ;

Place

```
# Bouton placé de manière absolue
button1 = ttk.Button(frame, text="Bouton 1")
# Positionnement absolu (50, 50)
button1.place(x=50, y=50)

# Bouton avec position relative à la taille de la fenêtre
button2 = ttk.Button(frame, text="Bouton 2")
# Centré à 50% de la largeur et hauteur
button2.place(relx=0.5, rely=0.5, anchor="center")
```

Place



Principaux widgets

Principaux widgets

Nous allons voir quelques widgets de base de `tkinter`, en mettant l'accent sur les plus récents (et mieux adaptés aux OS) de `ttk`.

Nous les présenterons surtout par des petits exemples. Une liste exhaustive des widgets et de leurs options serait fastidieuse (et inutile). Les meilleurs compagnons d'un développeur de GUI est la documentation.

L'idée ici est surtout de donner un aperçu de ce qui est possible, et de vous donner envie d'explorer par vous-même.

Principaux widgets

Pour ne pas alourdir les slides, ceux-ci ne contiennent que des morceaux de code illustrant un widget. Il est supposé que ces morceaux de code sont ajoutés dans un code similaire à celui-ci :

```
from tkinter import *
from tkinter import ttk

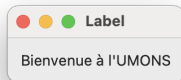
root = Tk()
root.title("Titre du widget illustré")
mainframe = ttk.Frame(root, padding=10)
mainframe.grid()
(...)
root.mainloop()
```

De plus, une vraie application GUI organiserait cela en utilisant la POO. Nous donnerons un exemple complet à la fin.

Label

- Un `ttk.Label` est un élément qui affiche du texte et/ou une image ;
- Il est utilisé pour afficher des instructions ou des informations ;
- Il n'est pas interactif (mais peut être modifié, comme nous l'avons fait dans l'exemple introductif).

```
label = ttk.Label(mainframe, text="Bienvenue à l'UMONS")
```



Label

On peut associer un `ttk.Style` à un label, pour changer son apparence. Le `.TLabel` est ajouté pour indiquer que le style s'applique à un label.

```
style = ttk.Style()
style.configure("UMonsRedBig.TLabel", foreground="#bf003a",
               font=("Arial", 28, "bold"), justify="center")
label = ttk.Label(mainframe, text="Bienvenue à l'UMONS",
                  style="UMonsRedBig.TLabel")
```



Label

Un `ttk.Label` peut contenir du texte (via l'option `text`) ou une image (via l'option `image`), ou les deux (en précisant via `compound` la position de l'image par rapport au texte).

Une image (de type `.png`) est instanciée via `PhotoImage`.

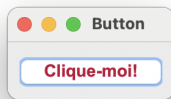
```
logoUmons = PhotoImage(file="umons.png")
label = ttk.Label(mainframe, text="Bienvenue à l'UMONS",
                  style="UMonsRedBig.TLabel",
                  image=logoUmons, compound="top")
```



Button

- Un `ttk.Button` est un élément interactif qui déclenche une action lorsqu'il est cliqué ;
- Il est utilisé pour déclencher des actions, comme appeler une fonction, ouvrir une fenêtre, etc. (via l'option `command`) ;
- On peut lui appliquer un style, il peut contenir du texte ou une image, comme pour les labels.

```
style = ttk.Style()
style.configure("UMonsRed.TButton", foreground="#bf003a",
               font=("Arial", 14, "bold"))
btn1 = ttk.Button(mainframe, text="Clique-moi!",
                  command=lambda: print("Clic!"),
                  style="UmonsRed.TButton")
```

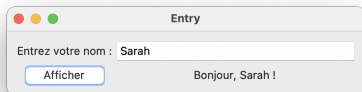


Entry

- Un `ttk.Entry` est un élément interactif qui permet à l'utilisateur de saisir du texte ;
- Il est utilisé pour récupérer des informations de l'utilisateur, comme un nom, un mot de passe, etc. via la méthode `get()`.

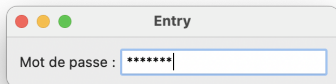
```
def afficher_nom(entry, label):  
    nom = entry.get()  
    label.config(text=f"Bonjour, {nom} !")  
    (...)  
label_instruction = ttk.Label(mainframe, text="Entrez votre nom :")  
label_instruction.grid(column=0, row=0)  
entry_nom = ttk.Entry(mainframe, width=30)  
entry_nom.grid(column=1, row=0)  
  
bouton_afficher = ttk.Button(mainframe, text="Afficher",  
                             command=lambda: afficher_nom(entry_nom, label_resultat))  
bouton_afficher.grid(column=0, row=1)  
label_resultat = ttk.Label(mainframe, text="")  
label_resultat.grid(column=1, row=1)
```


Entry



Une des options de `ttk.Entry` est `show`, qui permet de masquer les caractères saisis (e.g., pour un mot de passe).

```
entry_mdp = ttk.Entry(mainframe, width=30, show="*")
```



Un `tk.Text` permet d'avoir une entrée sur plusieurs lignes (voir chapitre suivant).

StringVar

Un `StringVar` est une variable spéciale dans Tkinter utilisée pour lier une valeur de type `str` à un widget comme un `Entry` ou un `Label` (via l'option `textvariable`).

Cela permet de mettre à jour automatiquement l'affichage du widget lorsque la valeur de la variable change, ou inversement, de récupérer la valeur saisie par l'utilisateur dans un widget.

Méthodes importantes des `StringVar` :

- `get` : récupère la valeur actuelle de la variable ;
- `set(value)` : modifie la valeur de la variable.

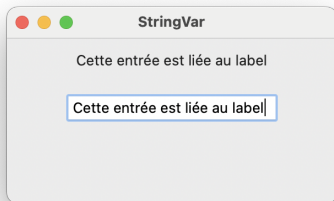
Il y a des équivalents pour les entiers (`IntVar`), les booléens (`BooleanVar`) et les réels (`DoubleVar`).

StringVar

```
string_var = StringVar()

label = ttk.Label(frame, textvariable=string_var)
label.pack(pady=10)

entry = ttk.Entry(frame, textvariable=string_var)
entry.pack(pady=10)
```

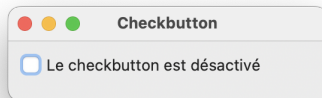
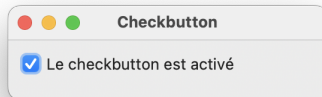


Checkbox

- Un `ttk.Checkbutton` est un élément interactif qui permet à l'utilisateur de cocher ou décocher une case ;
- Il est utilisé pour des options qui peuvent être activées ou désactivées.

```
def afficher_etat(cb, label):  
    etat = "activé" if cb.instate(["selected"]) else "désactivé"  
    label.config(text=f"Le checkbutton est {etat}")  
    (...)
  
label = ttk.Label(mainframe, text="")  
label.grid(column=1, row=0)  
check = ttk.Checkbutton(mainframe,  
                        command=lambda: afficher_etat(check, label))  
check.grid(column=0, row=0)
```

Checkbox

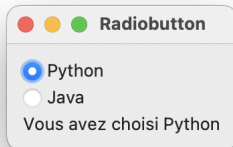


Radiobutton

- Un `ttk.Radiobutton` est un élément interactif qui permet à l'utilisateur de choisir une option parmi plusieurs ;
- Il est utilisé pour des options mutuellement exclusives.

```
def afficher_choix():
    choix = var.get()
    label.config(text=f"Vous avez choisi {choix}")
    (...)
var = StringVar()
var.set("Python")
label = ttk.Label(mainframe, text="")
label.grid(column=1, row=0)
radio1 = ttk.Radiobutton(mainframe, text="Python", variable=var,
                          value="Python", command=lambda: afficher_choix())
radio1.grid(column=0, row=0, sticky="W")
radio2 = ttk.Radiobutton(mainframe, text="Java", variable=var,
                          value="Java", command=lambda: afficher_choix())
radio2.grid(column=0, row=1, sticky="W")
```

Radiobutton



Il existe encore beaucoup d'autres widgets, que nous ne détaillerons pas ici. Pour les découvrir, consultez la documentation ou attendez le chapitre suivant où nous demanderons aux outils IA de nous aider.

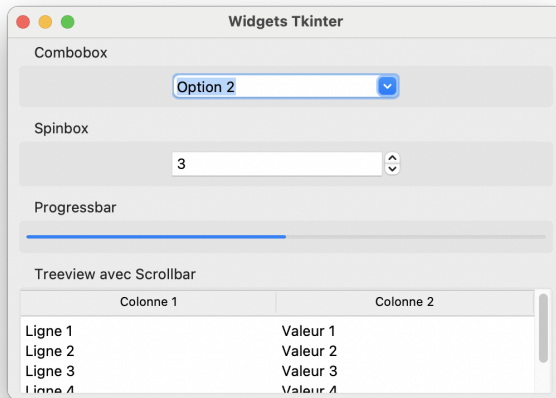
Tout ce que vous avez l'habitude de voir dans une GUI est possible, il suffit de chercher un peu.

Autres widgets utiles

- `ttk.Combobox` : Menu déroulant avec options sélectionnables.
- `ttk.Spinbox` : Champ numérique avec flèches pour incrémenter/décrément.
- `ttk.Progressbar` : Barre de progression.
- `ttk.Treeview` : Tableau pour afficher des données sous forme de liste/arborescence.
- `ttk.Notebook` : Onglets pour organiser des éléments.
- `tk.PanedWindow` : Divise une fenêtre en plusieurs parties redimensionnables.
- `tk.Scrollbar` : Barre de défilement.
- `tk.Message` : Message affiché dans une fenêtre.
- ...

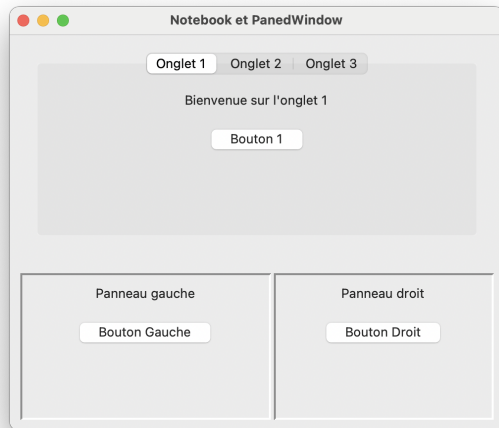
Notez que certains widgets sont dans le module `tkinter` (préfixé ici par `tk`), d'autres dans `ttk`.

Autres widgets utiles



Code `other_widgets1.py` disponible sur Moodle

Autres widgets utiles



Code `other_widgets2.py` disponible sur Moodle

Menus

Menus

Les menus sont des éléments importants d'une GUI.

- Un menu est composé de plusieurs éléments, qui peuvent être des commandes, des sous-menus, des séparateurs, etc ;
- Les menus sont créés via `Menu(parent)` (tk standard) où `parent` est une fenêtre ou un autre menu ;
- Les éléments d'un menu peuvent être des commandes (e.g., `add_command`), des sous-menus (e.g., `add_cascade`), des séparateurs (e.g., `add_separator`), etc ;
- Les commandes peuvent être associées à des fonctions via l'option `command` ;
- Les commandes peuvent être associés à des raccourcis clavier via l'option `accelerator`.

Menus

```
menu_bar = Menu(root)

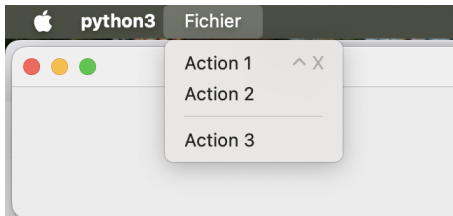
# Menu "Fichier"
fichier_menu = Menu(menu_bar, tearoff=0)
    # tearoff=0 pour enlever la barre de séparation
fichier_menu.add_command(label="Action 1", command=action_1,
                        accelerator="Ctrl+x")
fichier_menu.add_command(label="Action 2", command=action_2)
fichier_menu.add_separator()
fichier_menu.add_command(label="Action 3", command=action_3)

# Ajout du menu à la barre de menus
menu_bar.add_cascade(label="Fichier", menu=fichier_menu)

# Appliquer la barre de menu à la fenêtre principale
root.config(menu=menu_bar)

# Lier le raccourci clavier Ctrl+X à l'action 1
root.bind('<Control-x>', lambda event: action_1())
```

Menus



On reviendra sur le `bind` utilisé pour le raccourci clavier, dans la section sur les événements ([events](#)).

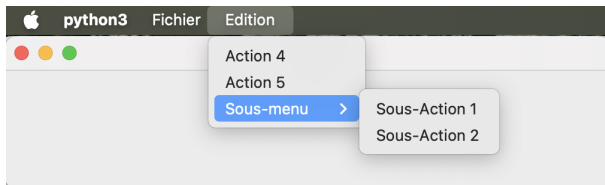
Menus

```
(...)
# Menu "Edition" avec sous-menu
edition_menu = Menu(menu_bar, tearoff=0)
edition_menu.add_command(label="Action 4", command=action_4)
edition_menu.add_command(label="Action 5", command=action_5)

# Sous-menu dans "Edition"
sous_menu = Menu(edition_menu, tearoff=0)
sous_menu.add_command(label="Sous-Action 1",
                      command=lambda: print("Sous-Action 1 choisie"))
sous_menu.add_command(label="Sous-Action 2",
                      command=lambda: print("Sous-Action 2 choisie"))
edition_menu.add_cascade(label="Sous-menu", menu=sous_menu)

# Ajout des menus à la barre de menu
menu_bar.add_cascade(label="Fichier", menu=fichier_menu)
menu_bar.add_cascade(label="Edition", menu=edition_menu)
(...)
```

Menus



Canvas

Canvas

Un Canvas (tk standard) est un élément qui permet de dessiner des formes.

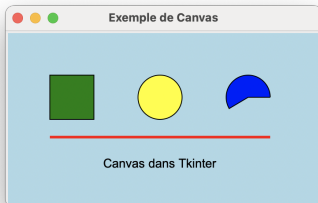
Méthodes principales :

- `create_line(x1, y1, x2, y2)` : crée une ligne entre les points (x1, y1) et (x2, y2) ;
- `create_rectangle(x1, y1, x2, y2)` : crée un rectangle délimité par les points (x1, y1) et (x2, y2) ;
- `create_oval(x1, y1, x2, y2)` : crée un ovale délimité par les points (x1, y1) et (x2, y2) ;
- `create_arc(x1, y1, x2, y2, start, extent)` : crée un arc de cercle délimité par les points (x1, y1) et (x2, y2), avec un angle de départ `start` et une étendue `extent` ;
- `create_text(x, y, text)` : crée un texte à la position (x, y).

Canvas

```
canvas = Canvas(root, width=400, height=300, bg="lightblue")
canvas.pack()

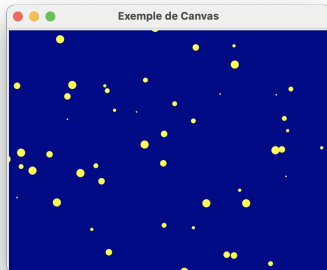
canvas.create_rectangle(50, 50, 100, 100, fill="green", outline="black")
canvas.create_oval(150, 50, 200, 100, fill="yellow", outline="black")
canvas.create_arc(250, 50, 300, 100, start=0, extent=210, fill="blue")
canvas.create_line(50, 120, 300, 120, fill="red", width=3)
canvas.create_text(175, 150, text="Canvas dans Tkinter",
                  font=("Arial", 14), fill="black")
```



Nuit étoilée avec Canvas

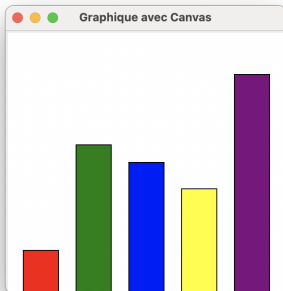
```
import random
canvas = Canvas(root, width=400, height=300, bg="darkblue")
canvas.pack()

for _ in range(50):
    x = random.randint(0, 400)
    y = random.randint(0, 300)
    size = random.randint(1, 5)
    canvas.create_oval(x-size, y-size, x+size, y+size,
                      fill="yellow", outline="")
```

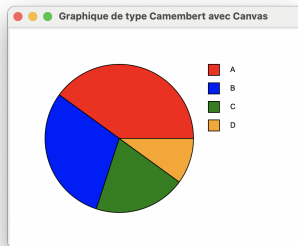


Histogramme avec Canvas

```
colors = ["red", "green", "blue", "yellow", "purple"]
data = [50, 170, 150, 120, 250]
for i, value in enumerate(data):
    canvas.create_rectangle(i * 60 + 20, 300,
                           i * 60 + 60, 300 - value, fill=colors[i])
```



Camembert avec Canvas



Démonstration du code en séance.

Animation avec Canvas

On peut créer des animations en déplaçant les éléments du canvas grâce à la méthode `after` d'une fenêtre qui lance une fonction après un certain délai (en *ms*).

```
class AnimatedCircle:
    def __init__(self, root, canvas):
        self.root = root
        self.canvas = canvas
        self.circle_id = self.canvas.create_oval(-5, 45, 5, 55,
                                                fill="yellow", outline="")

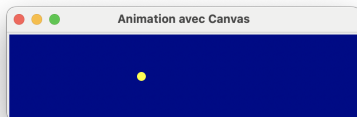
        self.move_circle()

    def move_circle(self):
        # Déplace le cercle de 2 pixels sur l'axe X
        self.canvas.move(self.circle_id, 2, 0)
        # Si le cercle a atteint le bord, et le remettre à gauche
        coords = self.canvas.coords(self.circle_id)
        if coords[2] >= 400:
            self.canvas.coords(self.circle_id, -5, 45, 5, 55)
            self.root.after(20, self.move_circle)

(...)
```

Animation avec Canvas

```
canvas = Canvas(root, width=400, height=100, bg="darkblue")  
canvas.pack()  
  
animated_circle = AnimatedCircle(root, canvas)
```



Démonstration de l'animation en séance,
ainsi que d'un ciel étoilé avec des étoiles filantes
(disponible sur Moodle).

Events

Events

Les événements (**events**) permettent d'interagir avec la GUI en capturant des actions de l'utilisateur, comme un clic de souris, une pression sur une touche du clavier ou le déplacement de la souris.

Dans Tkinter, les événements sont gérés avec le **binding** (`bind`) qui associe une action (`event`) à une fonction (`handler`).

Syntaxe : `widget.bind("type événement", handler)`.

- le premier argument est une chaîne de caractères prédéfinie qui indique le type d'événement à capturer (e.g., "`<Button-1>`" pour un clic gauche de souris, "`<KeyPress>`" pour une pression de touche, etc.) ;
- le second est une fonction qui doit accepter un argument `event` qui contient des informations sur l'événement en fonction de son type (e.g., position de la souris, touche enfoncée, etc.) et qui lui sera fourni automatiquement.

Events

```
def on_click(event):  
    print("Bouton cliqué !")  
  
btn = ttk.Button(root, text="Clique-moi")  
btn.pack(pady=20)  
btn.bind("<Button-1>", on_click)
```

Démonstration en séance.

Events

Principaux événements :

Événement	Description
<Button-1>	Clic gauche de la souris
<Button-2>	Clic du bouton du milieu
<Button-3>	Clic droit de la souris
<Double-Button-1>	Double-clic gauche
<Enter>	La souris entre dans le widget
<Leave>	La souris quitte le widget
<Motion>	Déplacement de la souris
<KeyPress>	Une touche du clavier est pressée
<KeyRelease>	Une touche du clavier est relâchée
<FocusIn>	Le widget gagne le focus clavier ¹
<FocusOut>	Le widget perd le focus clavier

1. Par exemple, en cliquant sur un champ `Entry` ou en naviguant avec `Tab`.

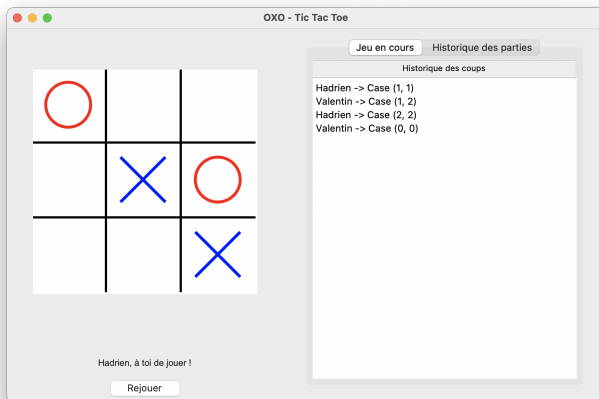
Events



Démonstration en séance du code `events.py` disponible sur Moodle.

Un exemple complet

Jeu d'OXO



Code `oxo.py` disponible sur Moodle